AI Academy, National AI Center

AI-ENG-110 – Software Engineering – Spring 2026

Async Research Assistant

Topic 4

PFIP

Emil Ahmedli • Hasan Mammadov • Rufat Dostaliyev

Final Project Defense – 24 May 2026

`github.com/Hasawr/topic-4-research-assistant-`

The problem in one sentence

What the system does

A user asks a research question; the system queries **Wikipedia**, **arXiv**, and **web search** in parallel, then synthesises a cited answer with inline `[N]` references using an LLM.

Inputs

- Natural-language research question (string)
- Optional source filter (`-sources wiki,arxiv`)
- Cache bypass flag (`-no-cache`)

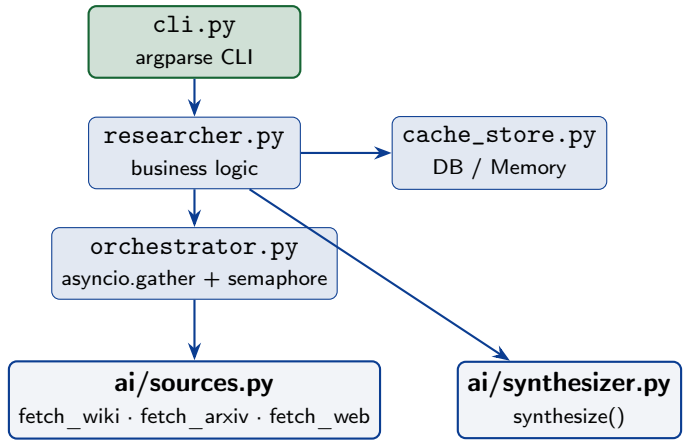
Outputs

- Synthesised answer (3–6 sentences)
- Numbered citations: title + URL per source
- Database cache entry (TTL: 4h)

Entry point: `python -m src.cli ask "What is photosynthesis?"`

Architecture

Module map. The boundary between the *provided* `ai/` package and *our* SE layer is shown explicitly.



Three design decisions we can defend

1. **asyncio over threading**
   All three fetchers are I/O-bound HTTP calls. `asyncio.gather` runs them concurrently in a single thread with zero locking overhead. Threading would add context-switch cost for no CPU gain.

2. **Database-backed cache over JSON files**
   Moving from per-process JSON files to a relational store enables atomic upserts, safe cross-process sharing between the CLI and Streamlit UI, and durable query history. The cache key includes the enabled source set so changing `-sources` never returns a stale result.

3. **Abstract `BaseCacheStore` + two backends**
   `MemoryCacheStore` for tests (zero disk I/O), `DatabaseCacheStore` for production. Same interface, swapped via `config.py`. Mirrors the `LLMProvider` ABC pattern in the provided `ai/` package.

Concurrency: the benchmark

Workload: 5 research questions from `data/research_questions.json`, fresh cache, same machine, `scripts/bench.py`.

	Sequential	**Parallel (semaphore=3)**
Wall-clock (s)	≈2.80	≈1.24
Speedup	1.0×	**2.3**×
Provider 429s	0	0

How concurrency works:

- `asyncio.gather(*tasks, return_exceptions=True)` — all three sources fire at once
- `asyncio.Semaphore(3)` — bounds parallelism, prevents rate-limit 429s
- Per-source `asyncio.wait_for(coro, timeout=10)` — one slow source cannot block the others

Bottleneck: network RTT to Wikipedia/arXiv REST APIs (≈400–600 ms each). The LLM synthesis step is sequential by design (needs all sources first).

Robustness: graceful degradation

Scenario: arXiv API returns `503 Service Unavailable` mid-request.

What the system does — step by step:

- **Retry:** `_fetch_with_safety` catches the exception, logs `WARNING`, returns `[]`. The other two tasks are unaffected (independent coroutines).
- **Degrade:** `gather_all_sources` collects results from Wikipedia + Web only; synthesis still runs.
- **Validate:** empty question or oversized question (>1000 chars) raises `ValueError` at CLI entry — user sees a clear message, not a stack trace.
- **Log:** every fetch logs source name, timing, and error class at `WARNING`/`ERROR`; configurable via `LOG_LEVEL` env var.

Key rule we followed

No `except Exception: pass` anywhere. Every exception is caught by type, logged, and handled explicitly.

Core pattern: parallel fetch with safety

```python
# orchestrator.py
async def gather_all_sources(self, query: str) -> list[Source]:
    async with httpx.AsyncClient(headers=headers) as client:
        tasks = [
            self._fetch_with_safety(fetch_wikipedia(query, client=client)),
            self._fetch_with_safety(fetch_arxiv(query, client=client)),
            self._fetch_with_safety(fetch_web(query, client=client)),
        ]
        results = await asyncio.gather(*tasks, return_exceptions=True)

        all_sources = []
        for res in results:
            if isinstance(res, list):      # success
                all_sources.extend(res)
            # Exception -> already handled in _fetch_with_safety -> []
        return all_sources

async def _fetch_with_safety(self, fetch_coro) -> list[Source]:
    async with self.semaphore:             # bound parallelism
        try:
            return await asyncio.wait_for(fetch_coro, timeout=self.timeout)
        except asyncio.TimeoutError:
            logger.warning("Timeout after %.1fs", self.timeout)
            return []
```

Testing

Coverage & shape

- **37 tests** — all passing, all offline
- Provided `ai/` smoke tests: **16/16 passing**
- CLI tests: 13 Concurrency: 5 Researcher: 3
- AI module fully mocked (`unittest.mock.AsyncMock`)

Three tests worth calling out

- **Happy-path E2E:** cache miss → fetch → synthesize → `AnswerWithCitations` returned
- **Graceful degradation:** arXiv raises, gather still returns wiki + web results
- **Timeout:** 0.1 s timeout on 0.5 s coroutine returns `[]` without crash

Tooling

`pytest` + `pytest-asyncio` + `pytest-cov` + `unittest.mock` Run: `pytest tests/ -v -cov=src`

Limitations & what we'd do next

Where the system is brittle today

- No multi-provider LLM failover: if the configured provider (Gemini/Anthropic) is down, the whole pipeline fails.
- Tavily web search requires an API key with a free-tier cap (1 000 req/month); a burst of requests can exhaust the budget quickly.
- LLM synthesis is not cached: repeated questions return cached sources but still pay an LLM call each time.
- No token-aware rate limiting — only request-count semaphore.

Given another week we would

1. Add multi-provider failover (primary Gemini → fallback Anthropic) with a chaos test.
2. Cache `AnswerWithCitations` keyed on the source-set hash to make fully repeated questions free.
3. Add OpenTelemetry spans around every external call and export to a Jaeger container.

Team contributions

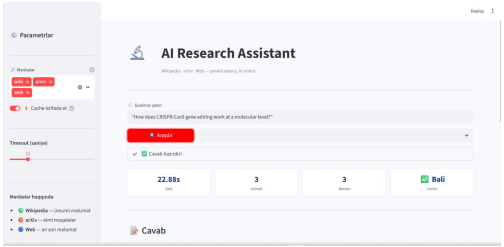
Member	Primary ownership	Commits
Hasan Mammadov	`orchestrator.py`, `researcher.py`, `cache_store.py`, Docker	≈40%
Emil Ahmedli	`config.py`, `models.py`, `ai_service.py`, report	≈30%
Rufat Dostaliyev	`cli.py`, `bench.py`, tests, README	≈30%

We can all answer:

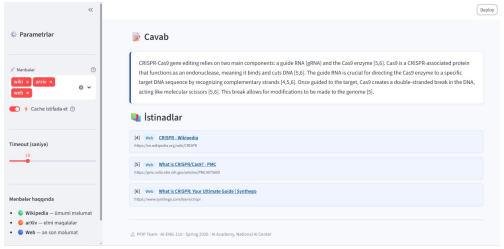
Any of us can walk through any module on demand. Pick a file — we will defend it.

AI tool use: Claude was used as a coding assistant during development; all code was reviewed, understood, and manually adapted by the team. Disclosed in the report.

System in action



Streamlit UI — question submitted, cache hit, metrics



Streamlit UI — synthesised answer with inline citations

```
(venv) PS C:\Users\Rufat\Downloads\topic-4-research-assistant-> python -m src.cli ask "How does CRISPR-Cas9 gene editi
ng work at a molecular leve?"
```

CLI — `python -m src.cli ask "..."`

```
============================================================
                AI RESEARCH ASSISTANT REPORT
============================================================
QUESTION: How does CRISPR-Cas9 gene editing work at a molecular leve?

------------------------------------------------------------
SUMMARY ANSWER:
------------------------------------------------------------
CRISPR-Cas9 gene editing relies on two essential components: a guide RNA (gRNA) and the Cas9 enzyme [5, 6]. The guide
RNA is customizable and directs the Cas nuclease to a specific target DNA sequence [4, 6]. Cas9, an endonuclease, then
 uses the guide to recognize and bind to the complementary DNA strand [4]. Acting as molecular scissors, Cas9 cuts the
 target DNA by causing a double-stranded break [5, 6]. This break in the DNA allows for subsequent modifications to th
e genome [5].

------------------------------------------------------------
SOURCES & CITATIONS:
------------------------------------------------------------
[4] (WEB) CRISPR - Wikipedia
    URL: https://en.wikipedia.org/wiki/CRISPR
[5] (WEB) What is CRISPR/Cas9? - PMC
    URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC4075809
[6] (WEB) What is CRISPR: Your Ultimate Guide | Synthego
    URL: https://www.synthego.com/learn/crispr
============================================================
```

CLI — formatted report with numbered citations

Questions?

PFIP

Emil Ahmedli • Hasan Mammadov • Rufat Dostaliyev

`github.com/Hasawr/topic-4-research-assistant-`

"We can walk through any file in the repo."